



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №4
«Шаблони «Singleton», «Iterator», «Proxy»,
«State», «Strategy».»
Варіант - 4

Виконала:
Студентка групи ІА-22
Третякова Ельвіра

Перевірив:
Мякий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми.....	7
Реалізація шаблону за обраною темою.....	8
<i>Діаграма класів для паттерну Proxy.....</i>	<i>13</i>
<i>Проблема, яку допоміг вирішити шаблон Proxy.....</i>	<i>14</i>
<i>Переваги застосування паттерну Proxy</i>	<i>15</i>
<i>Посилання на GitHub</i>	<i>15</i>
Висновок	15

Тема: Шаблони «Singleton», «Iterator», «Proxy», «State», «Strategy».

Мета: ознайомитися з шаблонами проектування «Singleton», «Iterator», «Proxy», «State», «Strategy» та набуті практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

Варіант – 4: Графічний редактор (proxy, prototype, decorator, bridge, flyweight, SOA)
Графічний редактор повинен вміти створювати / редагувати растрові (або векторні на розсуд студента) зображення в 2-3 основних популярних форматах (bmp, png, jpg), мати панель інструментів для створення графічних примітивів, вибору кольорів, нанесення тексту, додавання найпростіших візуальних ефектів (ч/б растр, інфрачервоний растр, 2-3 на вибір учня), роботи з шарами.

Короткі теоретичні відомості

Шаблони проектування — це формалізовані рішення типових задач, які часто виникають при проектуванні інформаційних систем. Вони містять опис проблеми, оптимальний спосіб її вирішення та рекомендації щодо використання. Кожен шаблон має уніфіковану назву, що полегшує спілкування між розробниками.

Шаблони дозволяють повторно використовувати знайдені успішні рішення. Їх застосування починається з правильного моделювання предметної області, що допомагає чітко сформулювати задачу та обрати відповідний патерн.

Переваги використання шаблонів

Застосування шаблонів забезпечує низку переваг:

- Структурованість системи, що виділяє ключові елементи та зв'язки;
- Простота у вивченні моделі та наочність, не втрачаючи глибини;
- Підвищення гнучкості системи, її стійкості до змін і спрощення доопрацювань;
- Полегшення інтеграції систем.

Шаблони створюють спільний "словник" для команди розробників, спрощуючи комунікацію між ними.

Шаблони зменшують трудовитрати, час проектування та витрати на підтримку системи. Вони додають системі необхідні властивості, як-от гнучкість і адаптивність. Шаблони економічно вигідні, оскільки дозволяють уникнути зайвої розробки, спираючись на перевірені практики.

Водночас їх варто використовувати обґрунтовано, зважаючи на потреби проекту, а не як універсальний інструмент.

Шаблон «Singleton»

Структура:



Призначення патерну:

«Singleton» (Одинак) являє собою клас в термінах ООП, який може мати не більше одного об'єкта (звідси і назва «одинак»). Насправді, кількість об'єктів можна задати (тобто не можна створити більш n об'єктів даного класу). Даний об'єкт найчастіше зберігається як статичне поле в самому класі.

Переваги та недоліки:

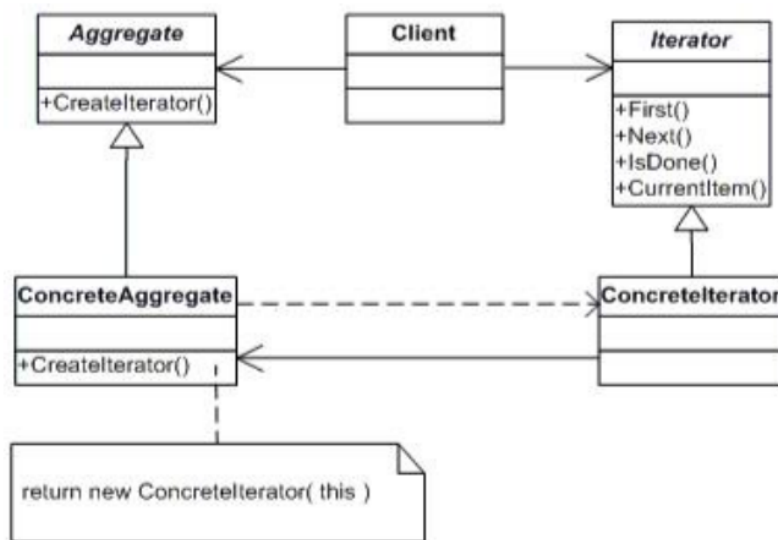
Однак слід зазначити, що в даний час «одинак» багато хто вважає т.зв. «анти-шаблоном», тобто поганою практикою проектування. Це пов'язано з тим, що «одинаки» представляють собою глобальні дані (як глобальна змінна), що мають стан. Стан глобальних об'єктів важко відслідковувати і підтримувати коректно; також глобальні об'єкти важко тестуються і вносять складність в програмний код (у всіх ділянках коду виклик в одне єдине місце з «одинаком»; при зміні підходу доведеться змінювати масу коду).

При цьому реалізація контролю доступу можлива за допомогою статичних змінних, замикань, мютексов та інших спеціальних структур.

- + Гарантує наявність єдиного екземпляра класу.
- + Надає до нього глобальну точку доступу.
- Порушує принцип єдиної відповідальності класу.
- Маскує поганий дизайн.

Шаблон «Iterator»

Структура:



Шаблонний ітератор містить:

- First() – установка покажчика перебору на перший елемент колекції;
- Next() – установка покажчика перебору на наступний елемент колекції;
- IsDone – булевське поле, яке встановлюється як true коли покажчик перебору досяг кінця колекції;
- CurrentItem – поточний об'єкт колекції.

Призначення:

«Iterator» (Ітератор) являє собою шаблон реалізації об'єкта доступу до набору (колекції, агрегату) елементів без розкриття внутрішніх механізмів реалізації. Ітератор виносить функціональність перебору колекції елементів з самої колекції, таким чином досягається розподіл

обов'язків: колекція відповідає за зберігання даних, ітератор - за прохід по колекції.

При цьому алгоритм ітератора може змінюватися - при необхідності пройти в зворотньому порядку використовується інший ітератор. Можливо також написання такого ітератора, який проходить список спочатку по парних позиціях (2,4,6-й елементи і т.д.), потім по непарних. Тобто, шаблон ітератор дозволяє реалізовувати різноманітні способи проходження по колекції незалежно від виду і способу представлення даних в колекції.

Переваги та недоліки:

Цей шаблон дозволяє уніфікувати операції проходження по наборам об'єктів для всіх наборів. Тобто, незалежно від реалізації (масив, зв'язаний список, незв'язаний список, дерево та ін.), кожен з наборів може використовувати будь-який з реалізованих ітераторів.

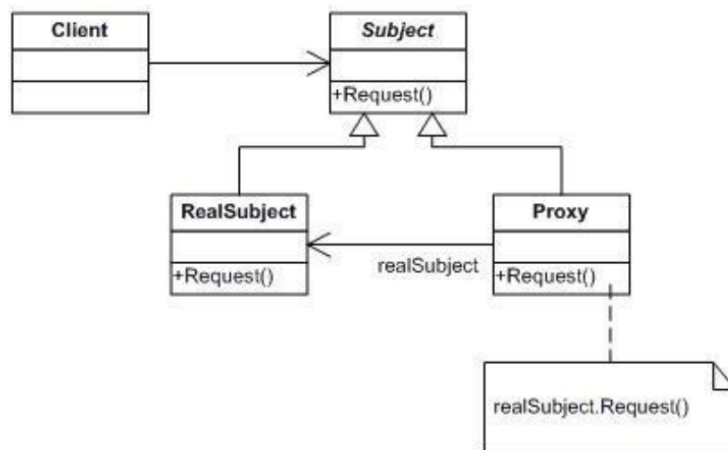
+ Дозволяє реалізувати різні способи обходу структури даних.

+ Спрощує класи зберігання даних

- Не виправданий, якщо можна обійтися простим циклом.

Шаблон «Proxy»

Структура:



«Проксу» (Проксі) - об'єкти є об'єктами-заглушками або двійниками/замінниками для об'єктів конкретного типу. Зазвичай, проксі об'єкти вносять додатковий функціонал або спрощують взаємодію з реальними об'єктами.

Проксі об'єкти використовувалися в більш ранніх версіях інтернет-браузерів, наприклад, для відображення картинки: поки картинка завантажується, користувачеві відображається «заглушка» картини, що виглядала так:



В якості іншого прикладу об'єктів заглушок можна привести аспектно-орієнтоване програмування. Наприклад, необхідно додати лог повідомлення при кожному доступі до окремого поля об'єктів певного класу. Є також обмеження, що змінювати програмний код класу не можна. В такому випадку, замість кожного такого об'єкта створюється проксі об'єкт, який при виклику цього методу переадресує виклик реальному об'єкту, але перед цим записує лог повідомлення.

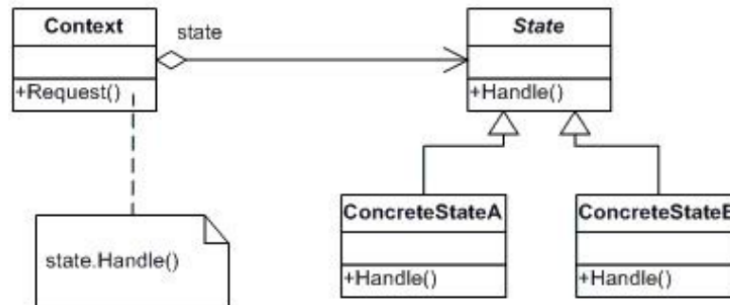
Проксі об'єкт повністю повторює визначення об'єкта, який він замінює (тобто всі доступні методи, поля і властивості); проксі об'єкт може також зберігати посилання на реальний об'єкт, в тому випадку, якщо він розширює функціональність, тобто «обгортає» реальний об'єкт.

Переваги та недоліки:

- + Дозволяє контролювати сервісний об'єкт непомітно для клієнта.
- + Може працювати, навіть якщо сервісний об'єкт ще не створений.
- Ускладнює код програми через введення додаткових класів.
- Збільшує час відгуку від сервісу.

Шаблон «State»

Структура:



Шаблон «State» (Стан) дозволяє змінювати логіку роботи об'єктів у випадку зміни їх внутрішнього стану. Наприклад, відсоток нарахованих на картковий рахунок грошей залежить від стану картки: Visa Electron, Classic, Platinum і т.д. Або обсяг послуг, які надані хостинг компанією, змінюється в залежності від обраного тарифного плану (стану членства - бронзовий, срібний або золотий клієнт). Реалізація даного шаблону полягає в наступному: пов'язані зі станом поля, властивості, методи і дії виносяться в окремий загальний інтерфейс (State); кожен стан являє собою окремий клас (ConcreteStateA, ConcreteStateB), які реалізують загальний інтерфейс.

Об'єкти, що мають стан (Context), при зміні стану просто записують новий об'єкт в поле state, що призводить до повної зміни поведінки об'єкта.

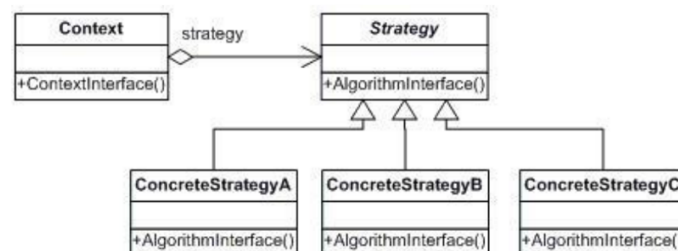
Це дозволяє легко додавати в майбутньому і обробляти нові стани, відокремлювати залежні від стану елементи об'єкта в інших об'єктах, і відкрито проводити заміну стану (що має сенс у багатьох випадках).

Переваги та недоліки:

- + Позбавляє від безлічі великих умовних операторів машини станів.
- + Концентрує в одному місці код, пов'язаний з певним станом.
- + Спрощує код контексту.
- Може невиправдано ускладнити код, якщо станів мало і вони рідко змінюються.

Шаблон «Strategy»

Структура:



Шаблон «Strategy» (Стратегія) дозволяє змінювати деякий алгоритм поведінки об'єкта іншим алгоритмом, що досягає ту ж мету іншим способом. Прикладом можуть служити

алгоритми сортування: кожен алгоритм має власну реалізацію і визначений в окремому класі; вони можуть бути взаємозамінними в об'єкті, який їх використовує.

Даний шаблон дуже зручний у випадках, коли існують різні «політики» обробки даних. По суті, він дуже схожий на шаблон «State» (Стан), проте використовується в абсолютно інших цілях - незалежно від стану об'єкта відобразити різні можливі поведінки об'єкта (якими досягаються одні й ті самі або схожі цілі).

Переваги та недоліки:

- + Гаряча заміна алгоритмів на льоту.
- + Ізолює код і дані алгоритмів від інших класів.
- + Відхід від спадкування до делегування.
- + Реалізує принцип відкритості / закритості.
- Ускладнює програму за рахунок додаткових класів.
- Клієнт повинен знати, в чому полягає різниця між стратегіями, щоб вибрати відповідну.

Реалізація не менше 3-х класів відповідно до обраної теми

Структура проекту з реалізованими класами зображена на рисунку 1:

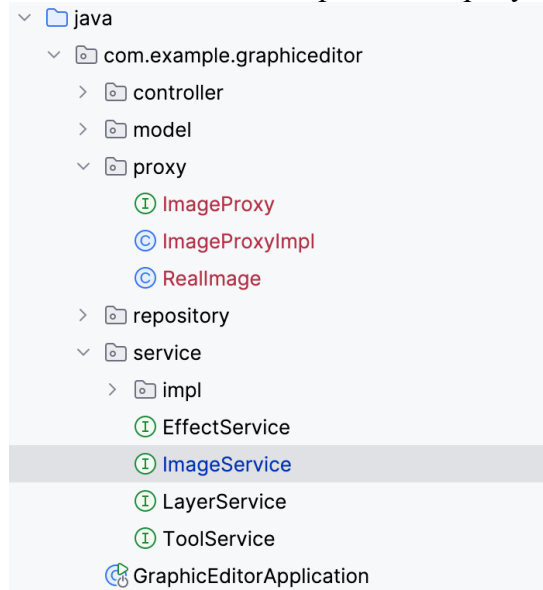


Рис.1 Структура проекту з реалізованими класами

У ході лабораторної роботи була реалізована система графічного редактора, в якій використано патерн Proxy для роботи із зображеннями. Основна мета — оптимізувати роботу із ресурсомісткими зображеннями, використовуючи проксі-об'єкт, який представляє реальний об'єкт лише тоді, коли це необхідно. У результаті виконано такі кроки:

1. Інтерфейс ImageProxy (Загальний інтерфейс):

Цей інтерфейс визначає загальні методи для роботи із зображенням:

- display() — метод для відображення зображення.
- getName() — метод для отримання назви зображення.

Інтерфейс забезпечує єдину точку доступу для всіх реалізацій (як для реального зображення, так і для проксі).

2. Клас *RealImage* (Реальний об'єкт):

Реалізує інтерфейс *ImageProxy* і є важким для завантаження об'єктом через ресурсомісткі операції.

Основна функціональність:

- Ініціалізація зображення відбувається з використанням конструктора, який симулює завантаження зображення з диску методом `loadFromDisk()`.

3. Клас *ImageProxyImpl* (Проксі-об'єкт):

Реалізує інтерфейс *ImageProxy* та виступає як "заступник" для класу *RealImage*. Основна мета проксі — відкласти створення ресурсоємного об'єкта до моменту, коли він справді потрібен.

Основна функціональність:

- Ліниве завантаження: Об'єкт класу *RealImage* створюється тільки під час першого виклику методу `display()`.

Реалізовані переваги:

1. Оптимізація використання ресурсів: Завантаження зображення відкладається до моменту першого використання.
2. Збереження цілісності інтерфейсу: Клієнт працює через єдиний інтерфейс *ImageProxy*, що спрощує взаємодію із системою.
3. Прозорість: Клієнт не знає, що працює з проксі-об'єктом, а не з реальним.
4. Економія пам'яті: У випадку великої кількості зображень проксі дозволяє завантажувати тільки необхідні з них.

Реалізація шаблону за обраною темою

Патерн Проху використовується для створення об'єкта-замісника, який контролює доступ до реального об'єкта. У нашому випадку ми створили проксі для роботи із зображеннями в графічному редакторі. Це дозволило оптимізувати завантаження реальних зображень із диска та відкладати їхнє створення до моменту першого звернення.

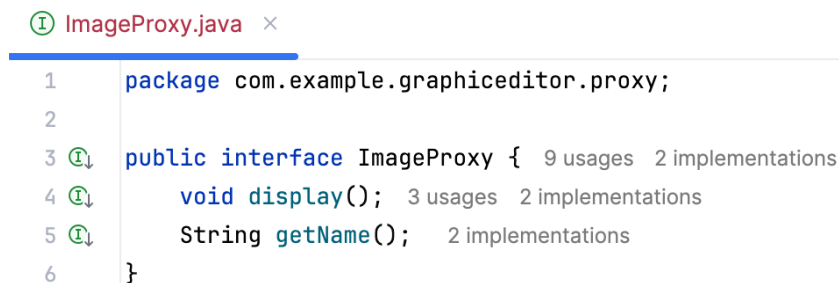
Опис реалізації:

1. Інтерфейс *ImageProxy*

Це загальний інтерфейс, який визначає методи для роботи із зображенням, незалежно від того, це проксі чи реальний об'єкт.

Методи:

- `display()` – метод для відображення зображення.
- `getName()` – метод для отримання назви зображення.



```
1 package com.example.graphiceditor.proxy;
2
3 public interface ImageProxy { 9 usages 2 implementations
4     void display(); 3 usages 2 implementations
5     String getName(); 2 implementations
6 }
```

Рис.2 Реалізація інтерфейсу *ImageProxy*

2. Клас RealImage (Реальний об'єкт)

Реалізує інтерфейс ImageProxy. Це "важкий" об'єкт, завантаження якого займає багато ресурсів (наприклад, час на завантаження файлу з диска). Основна функція класу:

- Завантаження зображення лише під час створення об'єкта.
- Відображення зображення через метод display().



```
1 package com.example.graphiceditor.proxy;
2
3 public class RealImage implements ImageProxy { 2 usages
4     private final String name; 4 usages
5     private final String format; 3 usages
6
7     public RealImage(String name, String format) { 1 usage
8         this.name = name;
9         this.format = format;
10        loadFromDisk();
11    }
12
13    private void loadFromDisk() { 1 usage
14        System.out.println("Завантаження зображення з диску: " + name + "." + format);
15    }
16
17    @Override 3 usages
18    public void display() {
19        System.out.println("Відображення зображення: " + name + "." + format);
20    }
21
22    @Override
23    public String getName() {
24        return name;
25    }
26 }
```

Рис. 3 Реалізація класу RealImage

Клас RealImage є реалізацією реального об'єкта, який містить повну логіку роботи із зображенням. Він реалізує інтерфейс ImageProxy, що визначає загальні методи для роботи із зображенням. Цей клас відповідає за зберігання інформації про зображення, його завантаження та відображення.

1. Поля класу

Клас має два приватні фінальні поля:

- name – назва зображення.
- format – формат файлу зображення (наприклад, .jpg, .png).

Ці поля ініціалізуються під час створення об'єкта класу RealImage.

Оскільки ці поля оголошені як final, їх значення не можуть змінюватися після ініціалізації.

2. Конструктор

Конструктор класу приймає два параметри:

- name – назва зображення.
- format – формат зображення.

Під час створення об'єкта викликається метод loadFromDisk(), який симулює ресурсомістке завантаження зображення з диска

3. Приватний метод loadFromDisk()

Цей метод використовується для симуляції завантаження зображення з диска. У реальних умовах тут би відбувалося читання файлу з диска, але в нашій реалізації це просто виведення повідомлення в консоль.

Цей метод викликається лише один раз під час створення об'єкта.

4. Метод display() (з інтерфейсу ImageProxy)

Цей метод відповідає за відображення зображення. Реалізація передбачає просте виведення повідомлення в консоль, яке імітує процес відображення зображення.

5. Метод getName() (з інтерфейсу ImageProxy)

Цей метод дозволяє отримати назву зображення. Він просто повертає значення поля name.

- *Призначення:* Клас RealImage реалізує всю функціональність роботи із зображенням, включаючи завантаження (в симуляції) та відображення. Він використовується в якості реального об'єкта в патерні Proxy.
- *Завдання:* Він забезпечує:
 1. Зберігання інформації про зображення.
 2. Завантаження зображення з диска при створенні об'єкта.
 3. Відображення зображення.
 4. Доступ до назви зображення.

Переваги класу RealImage

1. *Чітка структура:* Клас виділяє окремо функції для завантаження, відображення та отримання інформації про зображення.
2. *Інкапсуляція логіки завантаження:* Логіка завантаження прихована у приватному методі loadFromDisk(), що забезпечує зручність у використанні та безпеку.
3. *Відповідність принципу "єдиного обов'язку":* Клас відповідає лише за управління даними та поведінкою реального зображення.

Клас RealImage є основою для роботи патерну Proxy, оскільки саме цей клас обробляє всі запити, делеговані йому проксі-об'єктом.

3. Клас ImageProxyImpl (Проксі-об'єкт)

Цей клас також реалізує інтерфейс ImageProxy, але виступає замісником для RealImage. Його основна мета – відкладене завантаження реального зображення до моменту, коли це справді необхідно (виклик методу display()).

Основні функції:

- Зберігає інформацію про зображення (назва та формат).
- Завантажує реальний об'єкт лише за першого звернення.
- Делегує всі виклики методів реальному об'єкту.

```

1  package com.example.graphiceditor.proxy;
2
3  public class ImageProxyImpl implements ImageProxy { 3 usages
4      private final String name; 3 usages
5      private final String format; 2 usages
6      private RealImage realImage; 3 usages
7
8      public ImageProxyImpl(String name, String format) { 2 usages
9          this.name = name;
10         this.format = format;
11     }
12
13     @Override 3 usages
14     public void display() {
15         if (realImage == null) {
16             realImage = new RealImage(name, format);
17         }
18         realImage.display(); |
19     }
20
21     @Override
22     public String getName() {
23         return name;
24     }
25 }
26

```

Рис.4 Реалізація класу ImageProxyImpl

Цей код є реалізацією патерну проектування Проксу для роботи із зображеннями. Патерн Проксу використовується, щоб створити сурогатний об'єкт, який контролює доступ до реального об'єкта (в даному випадку до зображення).

Клас ImageProxyImpl реалізує інтерфейс ImageProxy і виступає проксі-об'єктом для роботи з реальним зображенням.

Поля:

1. `private final String name;`
Це ім'я зображення. Зберігається для ідентифікації зображення.
2. `private final String format;`
Формат зображення (наприклад, jpg, png тощо).
3. `private RealImage realImage;`
Поле для реального об'єкта зображення. Ініціалізується лише тоді, коли це дійсно потрібно (лінива ініціалізація).

Конструктор:

Конструктор приймає ім'я зображення та його формат. Ці дані зберігаються у відповідних полях і використовуються для створення реального зображення, коли це потрібно.

Метод `display()`:

- Перевіряє, чи `realImage` вже створений.

- Якщо ні (тобто він null), створює новий екземпляр класу RealImage з вказаним іменем і форматом. Це реалізує ліниву ініціалізацію.
- Викликає метод display() у реального зображення (realImage).

Метод getName():

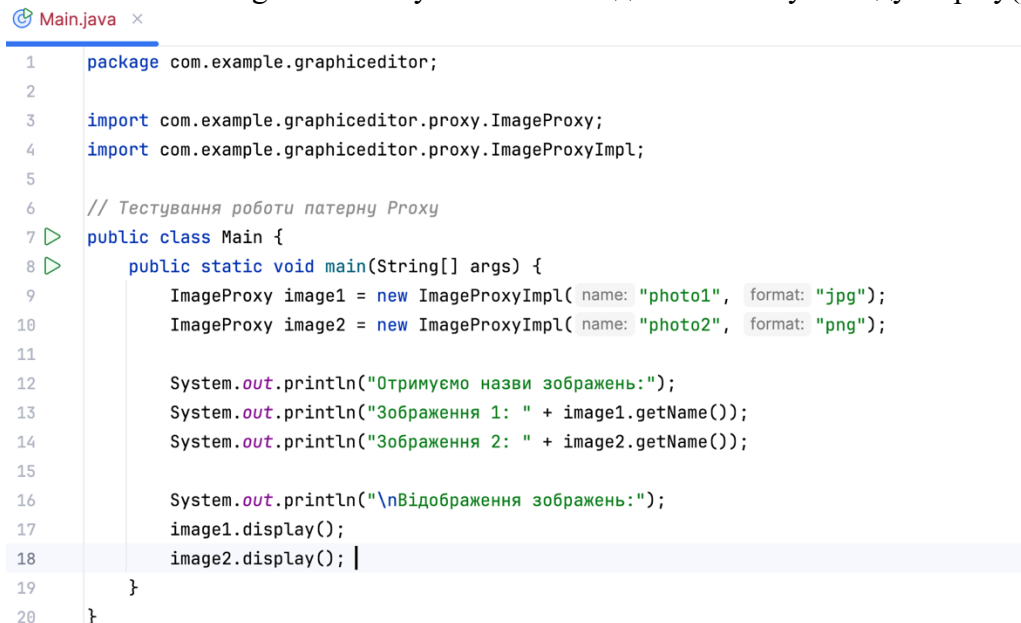
Цей метод повертає ім'я зображення. Це допоміжний метод, який може бути корисним для отримання метаданих.

Переваги реалізації:

1. Оптимізація ресурсів: Реальне зображення створюється лише тоді, коли це дійсно необхідно. Це може бути корисно, якщо зображення великі або їх завантаження займає час.
2. Контроль доступу: Проксі дозволяє контролювати доступ до реального зображення. Наприклад, можна додати перевірку прав доступу.
3. Прозорість: Для клієнта (ImageProxy) проксі та реальний об'єкт виглядають однаково.

4. Демонстрація роботи (Клієнтський код)

Для тестування роботи Proxu було створено клієнтський код, який взаємодіє із зображенням через проксі. У цьому коді ми створюємо об'єкт ImageProxyImpl, але реальний об'єкт RealImage завантажується лише під час виклику методу display().



```

1 package com.example.graphiceditor;
2
3 import com.example.graphiceditor.proxy.ImageProxy;
4 import com.example.graphiceditor.proxy.ImageProxyImpl;
5
6 // Тестування роботи патерну Proxu
7 public class Main {
8     public static void main(String[] args) {
9         ImageProxy image1 = new ImageProxyImpl( name: "photo1", format: "jpg");
10        ImageProxy image2 = new ImageProxyImpl( name: "photo2", format: "png");
11
12        System.out.println("Отримуємо назви зображень:");
13        System.out.println("Зображення 1: " + image1.getName());
14        System.out.println("Зображення 2: " + image2.getName());
15
16        System.out.println("\nВідображення зображень:");
17        image1.display();
18        image2.display();
19    }
20 }

```

Рис.5 Реалізація демонстраційної роботи

Цей код демонструє, як працює патерн Proxu, використовуючи клас ImageProxyImpl.

Основні моменти:

1. Створення проксі для зображень:

Об'єкти image1 і image2 створюються без негайного завантаження зображень (завдяки патерну Proxu).

2. Отримання метаданих:

Метод getName() повертає імена зображень без необхідності створювати їх реальні об'єкти.

3. Ліниве завантаження та відображення:

Реальні об'єкти ReallImage створюються та завантажуються з диска лише під час першого виклику методу display().

Суть:

- Імена зображень отримуються без створення реальних об'єктів.
 - Зображення завантажуються лише за потреби (під час відображення).
- Це оптимізує використання ресурсів, реалізуючи переваги патерну Proxy.

Результат виконання:

```
Run Main x
/Library/Java/JavaVirtualMachines/jdk-19.jdk/Contents/Home/bin/java ...
Отримуємо назви зображень:
Зображення 1: photo1
Зображення 2: photo2

Відображення зображень:
Завантаження зображення з диску: photo1.jpg
Відображення зображення: photo1.jpg
Завантаження зображення з диску: photo2.png
Відображення зображення: photo2.png
```

Рис.6 Результат виконання коду демонстраційної роботи

Діаграма класів для патерну Proxy

Діаграма класів, яка реалізовує патерн Proxy зображена на рисунку 7

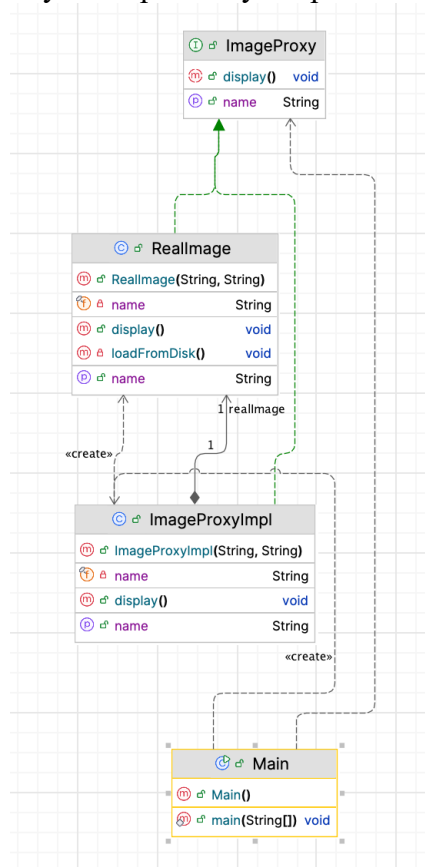


Рис. 7 Діаграма класів для патерну Proxy

Ця діаграма класів представляє структуру застосування шаблону Proxu для роботи з графічними зображеннями в рамках вашого графічного редактора.

1. Основні класи:

Інтерфейс ImageProxu

Задає контракт для роботи із зображенням через проксі.

Клас RealImage

Це клас, який відповідає за безпосередню роботу із зображенням (реалізація реального об'єкта).

Основні методи:

RealImage(String, String): Конструктор, який завантажує зображення з диска, використовуючи ім'я та формат.

display(): Метод для відображення зображення.

loadFromDisk(): Приватний метод для завантаження зображення з диска під час ініціалізації.

Клас ImageProxyImpl

Реалізує інтерфейс ImageProxu.

Діє як посередник між клієнтом і реальним зображенням (RealImage), забезпечуючи ліниве завантаження.

Клас Main

Головний клас для тестування роботи Proxu.

Демонструє створення проксі-об'єктів (ImageProxyImpl), отримання імен зображень та їх відображення.

2. Зв'язки між класами:

1. Інтерфейс ImageProxu:
 - Є абстракцією, яку реалізує клас ImageProxyImpl.
2. Клас ImageProxyImpl:
 - Зберігає посилання на RealImage для передачі викликів.
3. Клас RealImage:
 - Реалізує реальну поведінку роботи із зображеннями (завантаження та відображення).
4. Клас Main:
 - Взаємодіє лише з ImageProxu (не з RealImage безпосередньо), що забезпечує ізоляцію клієнта від реалізації.

Проблема, яку допоміг вирішити шаблон Proxu

Патерн Proxu у моєму випадку вирішив проблему оптимізації роботи з ресурсами та зменшення затримок у роботі програми.

1. Ліниве завантаження зображень:

Завантаження великих зображень з диска може бути ресурсомістким і вимагати значного часу. Патерн Proxu дозволив уникнути негайного завантаження, відкладаючи створення реального об'єкта RealImage до моменту, коли зображення дійсно потрібно відобразити (метод display()).

2. Зменшення затримок у роботі програми:

Коли програма взаємодіє з багатьма зображеннями, але не всі з них одразу необхідно завантажувати або відображати, Proxu допомагає забезпечити швидке реагування програми на дії користувача (наприклад, отримання метаданих, таких як назва зображення).

3. *Контроль доступу:*

Proxu виступає посередником, який контролює створення та виклик методів реального об'єкта. Це дозволяє додавати логіку перевірки, кешування або додаткових обмежень (хоча в цьому прикладі це поки не реалізовано).

4. *Зменшення споживання пам'яті:*

Об'єкти RealImage створюються тільки за потреби, що знижує використання оперативної пам'яті, особливо при роботі з великою кількістю зображень.

5. *Ізоляція клієнтського коду:*

Клієнт працює із загальним інтерфейсом ImageProxu, не знаючи деталей про те, як і коли відбувається завантаження або створення реального зображення. Це спрощує підтримку та розширення системи.

Патерн Proxu дозволив оптимізувати завантаження ресурсів, підвищити продуктивність програми та забезпечити гнучкість і прозорість у роботі з об'єктами зображень у графічному редакторі.

Переваги застосування патерну Proxu

Використання патерну Proxu у моєму графічному редакторі має такі переваги:

1. Ліниве завантаження зображень

- Зображення завантажуються в пам'ять лише тоді, коли це потрібно, наприклад, під час відображення на екрані. Це економить ресурси, особливо для великих файлів або багатьох зображень.

2. Оптимізація роботи з великими проектами

- У графічному редакторі, де одночасно може оброблятися багато шарів або зображень, Proxu дозволяє працювати лише з метаданими (ім'ям, форматом тощо) без завантаження самих даних. Це знижує навантаження на оперативну пам'ять і підвищує швидкість роботи.

3. Зменшення затримок

- Під час відкриття проекту всі зображення або шари додаються до редактора як проксі. Завантаження реальних даних відбувається лише під час їх відображення або редагування. Це значно пришвидшує час завантаження великих проектів.

4. Гнучке управління ресурсами

- Proxu дозволяє управляти доступом до зображень. Наприклад, можна реалізувати завантаження зображень у фоновому режимі або кешування часто використовуваних шарів.

Посилання на GitHub

Посилання на репозиторій: <https://github.com/Ellviraaaaaa/GraphicEditor.git>

Висновок

У даній лабораторній роботі я реалізувала патерн проектування Proxu для графічного редактора, що дозволило оптимізувати роботу із зображеннями шляхом впровадження механізму відкладеного завантаження та зменшення навантаження на систему. Завдяки цьому шаблону вдалося забезпечити контроль доступу до ресурсів і оптимізувати взаємодію з великими графічними файлами.

Застосування Proxu дозволило інкапсулювати логіку роботи із зовнішніми ресурсами та розділити функції представлення й обробки даних. Це забезпечило спрощення архітектури програми, підвищило її продуктивність і масштабованість.

